FREE PROMPT PACK · CLAUDE CODE · LOCAL AI

Build Jarvis *Fully* Offline

The hardware guide, three installs and six copy-paste prompts. Part One tells you which machine to buy, how much unified memory you actually need, and which AI models run best on a Mac mini, a Mac Studio or a Windows PC. Part Two puts three free engines on it: Ollama for the brain, VoiceStudio for the voice, Whisper for the ears. Part Three builds Jarvis on top with Claude Code: a 3D galaxy of your notes, a butler who answers from them in a cloned voice, hands-free listening, a memory that grows when you talk, and a drill that proves nothing leaves the machine. No subscriptions, no API bills, no data leaving your house.

ZUBAIR TRABZADA

AI WORKSHOP

NOVEMBER 2026

How to use this pack

What you need: a computer with 16 GB of memory or more (Part One tells you which one), Claude Code to do the building (pick Claude Opus 5.5 with /model), Google Chrome, Python 3, and a folder of markdown notes. Claude Code is the only thing here that touches the cloud, and only while it builds. Everything Jarvis runs on afterwards is free, open source and offline.

Set up in one line. Open a terminal, make an empty folder, and start Claude Code inside it: `mkdir my-jarvis && cd my-jarvis && claude`. Read Part One before you buy anything, do the three installs in Part Two, then paste Prompt 01.

1 **Memory is the whole game.**
An AI model has to fit in memory to run at all. On a Mac that is the unified memory; on a Windows PC it is the graphics card's VRAM, or unified memory on the new AMD AI chips. Part One turns that one number into a shopping decision.

2 **Claude Code builds it. Your computer runs it.**
The building happens in the cloud, once. What comes out is a folder of plain Python and HTML that runs on your machine with the three local engines. After the build, Jarvis never needs a key.

3 **Install first, prompt second.**
Prompts 01 to 03 expect Ollama, VoiceStudio and Whisper to already answer on your machine. Each install page ends with one command that proves it. Run it before you move on.

4 **Paste each prompt whole, unedited, the first time.**
Every prompt is a complete brief: what to build, the laws it must respect, and how to prove it works. The verification at the end of each one is where the reliability comes from.

5 **Clone only a voice you may use.**
Your own voice, or one you have permission for. The engines this pack pins are licensed for any use, including commercial. Cloning a famous voice for anything public is a legal problem, not a technical one.

6 **Test quietly.**
Test tabs open with ?mute=1 so a background tab never speaks. Every prompt that touches the voice or the ears keeps that law.

Part One · the hardware

Jarvis has three jobs running at once: a brain (the language model), a voice (the speech engine) and ears (Whisper). All three live in memory at the same time, and the brain is by far the biggest. So the question is never "which computer is fastest". It is "how much memory can the model see". Three machines cover everyone.

Mac mini	The always-on butler. Small, silent, sips power, and the M6 with 32 GB runs a genuinely good brain. The machine most people should buy.
Mac Studio	The home data centre. An M5 Ultra with 96 GB to 256 GB runs the 100B-class open models that were cloud-only a year ago, with the voice and the ears beside them.
Windows PC	Two roads. An NVIDIA card with 16 to 32 GB of VRAM is fast but small. A Ryzen AI Max+ 395 mini PC with 128 GB of unified memory is slower per word but fits far bigger brains.

Why unified memory matters. On a Mac and on the AMD AI chips, the CPU, the GPU and the neural engine share one pool of memory. The model loads into that pool once and every part of the chip can read it. On a classic Windows PC the model has to fit inside the graphics card's own VRAM; the system RAM does not count. A 5090 has 32 GB of VRAM. A Mac mini can be ordered with 32 GB. A Mac Studio goes to 512 GB. That single number decides the brain.

> **The rule of thumb.** A model stored at 4-bit needs about 0.6 GB of memory per billion parameters, and you keep roughly 30 percent of the memory free for the system, the context window, the voice engine and Whisper. So 16 GB runs a brain up to about 15B parameters, 32 GB up to about 35B, 128 GB up to about 140B, 256 GB up to about 300B. Buy memory. Everything else on the spec sheet is a distant second.

How much memory you actually need

Find your memory in the left column. The middle column is the biggest brain that fits comfortably with the voice and the ears loaded beside it. The right column is the honest description of how it feels to live with.

MEMORY	BIGGEST COMFORTABLE BRAIN	WHAT TO EXPECT
8 GB	Gemma 4 E4B (4B) or Gemma 2 2B	Small talk and simple lookups. Keep the real thinking for a cloud brain. Not recommended for a Jarvis you want to live with.
16 GB	**Qwen 3.5 9B** (about 6 GB)	The floor. A real assistant that answers from your notes and carries a personality. The base Mac mini and most laptops live here.
24 GB	Gemma 4 12B (about 8 GB) or gpt-oss-20b (21B, about 13 GB)	Noticeably smarter, better at following your rules, and Gemma 4 can see images.
32 GB	**Qwen 3.8 27B** (about 17 GB) or Gemma 4 26B A4B	The sweet spot. A brain that holds a long conversation, follows ten rules at once and lands the jokes. Mac mini M6 32 GB, or an RTX 5090.
64 GB	Gemma 4 31B (about 20 GB) plus a vision model, all resident	Everything from 32 GB, plus room for a second model and a huge context window without evicting the brain. Mac mini M5 Pro or Mac Studio M5 Max.
96 to 128 GB	**gpt-oss-120b** (117B, about 65 GB) or Llama 4 Scout (109B, about 67 GB)	Cloud-class open models at home. Mac Studio M5 Ultra 96 GB, M5 Max 128 GB, or a Ryzen AI Max+ 395 mini PC with 128 GB.
256 GB and up	Qwen 3.8 Flash-Next (125B plus 51B, about 105 GB)	The biggest open brains with the whole crew resident beside them: brain, vision, voice, ears, embeddings. Mac Studio M5 Ultra territory.

Two traps. First, the model that "fits" with 2 GB to spare does not fit: the context window grows as you talk, the voice engine needs its own few gigabytes, and the machine starts swapping to disk and everything crawls. Use the 30 percent rule. Second, a bigger brain is not always a better Jarvis. The 27B model follows a short rulebook better than the 120B one follows a long one. Prompt 04 is built for that.

Which models run best on which machine

Every model here is open-weight and free to download through Ollama. Sizes are 4-bit estimates; the exact tag is the name on the model's page at ollama.com/library. Pick the top row that fits your machine and start there.

MODEL	PARAMETERS	MEMORY	RUNS BEST ON
Qwen 3.5 9B	9B dense	about 6 GB	Any 16 GB machine. The brain this pack ships with. Hides its thinking unless you tell it not to think (Install 01 shows the switch).
Gemma 4 12B	12B dense	about 8 GB	24 GB Mac mini, 16 GB NVIDIA cards. Sees images, so Jarvis can look at a screenshot.
gpt-oss-20b	21B, 3.6B active	about 13 GB	24 to 32 GB machines. OpenAI's small open model; quick because only a slice of it fires per word.
Qwen 3.8 27B	27B dense	about 17 GB	**Mac mini M6 32 GB, RTX 5090.** The best brain that fits a mini. Apache 2.0 licence.
Gemma 4 26B A4B	26B, 4B active	about 18 GB	32 GB machines that want speed over depth. Mixture of experts, long 256K context.
Gemma 4 31B	31B dense	about 20 GB	64 GB and up. Fits on 32 GB but it is the ceiling there; comfortable on a 64 GB Mac.
gpt-oss-120b	117B, 5.1B active	about 65 GB	**Mac Studio 96 GB and up, Ryzen AI Max+ 128 GB.** OpenAI's big open model. Cloud-class reasoning at home.
Llama 4 Scout	109B, 17B active	about 67 GB	96 GB and up. Meta's open model, natively multimodal, very long context.
Qwen 3.8 Flash-Next	125B plus 51B n-gram, 6B active	about 105 GB	Mac Studio M5 Ultra 256 GB. The biggest open brain that still answers quickly, because only 6B fires per word.

Dense versus active. "27B dense" means all 27 billion parameters work on every word. "117B, 5.1B active" is a mixture-of-experts model: it must fit in memory whole, but only about 5 billion parameters fire per word, so it answers faster than its size suggests. For Jarvis, where the conversation needs to flow, a big mixture-of-experts brain is often the better buy than a mid-size dense one.

The buying guide

Three honest picks, one per budget. Prices are US list prices in late 2026 and they move; memory shortages pushed every one of these up this year, so check the store before you order. In every case, put the money into memory first, storage second, and ignore the CPU core count entirely.

THE PICK FOR MOST PEOPLE

Mac mini M6, 32 GB

from about $1,299 with 256 GB storage

- Runs Qwen 3.8 27B with the voice and the ears beside it.
- Silent, 5 inches square, lives on a shelf for years. Leave it on.
- Apple's MLX engines make the voice and Whisper fast without any setup.
- Order 32 GB. The 16 GB base model is a 9B-only machine and you will outgrow it in a month.
- Skip the storage upgrade: models live happily on an external SSD.

THE BIG BRAIN

Mac Studio M5 Ultra

from about $5,499 with 96 GB; 256 GB adds roughly $4,000

- 96 GB runs gpt-oss-120b and Llama 4 Scout. 256 GB runs Flash-Next plus the whole crew.
- The 128 GB option is the M5 Max chip, not the Ultra. It runs the same 100B-class models, a bit slower, for less money.
- 10Gb Ethernet and six Thunderbolt 5 ports: serve Jarvis to every device in the house.
- Only worth it if you want the biggest open brains. A mini runs a good Jarvis.

THE WINDOWS ROADS

NVIDIA card, or Ryzen AI Max+ 395

RTX 5090 cards about $4,500 alone in 2026; AI Max+ 128 GB mini PCs about $2,200 to $3,800

- A 16 GB card (5070 Ti, 5080) is a 9B to 12B machine. A 5090 with 32 GB matches the 32 GB mini and is quicker per word.
- A Ryzen AI Max+ 395 mini PC (Framework Desktop, GMKtec EVO-X2, Beelink GTR9 Pro, HP Z2 Mini) gives the GPU up to 96 GB: 100B-class brains in a box the size of a mini.
- VoiceStudio and Whisper both run on Windows with an NVIDIA card. On AMD they fall back to the CPU unless you enable ROCm.

What not to pay for. Faster CPU cores (the model runs on the GPU and the memory). More than 1 TB of internal storage (a 2 TB external SSD is cheaper than Apple's upgrade and holds thirty models). A display (Jarvis runs headless; you talk to him from any browser on the network). And never buy the base memory of anything on this page because "it was on sale".

Part Two · the installs

Three free engines, each one a desktop app or a Python package, each one answering on a local port that the prompts in Part Three will talk to. Nothing here needs an account. Do them in order; each takes about ten minutes plus the download. Mac and Windows commands are both on each page.

The brain	Ollama runs open models on your machine and exposes them through the same style of endpoint the cloud brains use, so Jarvis cannot tell the difference.
The voice	VoiceStudio clones a voice from a 10 to 20 second clip and speaks any text through it, with engines licensed for any use on both Mac and Windows.
The ears	Whisper turns your speech into text on your own chip. Two sizes: a large one for your commands and a small one that only listens for the wake word.

Two laws hold across all three. Pin everything: the model, the engine, the version. "Latest" is how a working Jarvis breaks on a Tuesday. And prove each install with the one command at the bottom of its page before you touch the next one; Claude Code cannot fix an engine that was never there.

YOUR MEMORY	BRAIN TO PULL FIRST	THEN TRY
16 GB	**qwen3.5:9b**	Nothing bigger fits comfortably. Make this one lean with Prompt 04.
24 GB	gemma4:12b	gpt-oss-20b for quicker replies.
32 GB	**Qwen 3.8 27B**	Gemma 4 26B A4B, and keep qwen3.5:9b on the shelf for small talk.
96 GB and up	gpt-oss-120b	Llama 4 Scout; Flash-Next if you have 256 GB.

INSTALL 01 · THE BRAIN

Ollama and a local model

Best for: the first install. Ten minutes, one download, and a brain that costs nothing per reply.

1 Download Ollama from **ollama.com/download** (Mac, Windows, Linux), open it once, and let it sit in your menu bar or system tray. It serves models on port 11434. On Windows with an NVIDIA card it uses the GPU automatically; on a Ryzen AI Max+ machine, set the GPU memory share to its maximum in the BIOS first so the model has room.

2 Pull the brain for your memory size from the table on the previous page. The pull is a one-time download of a few gigabytes; the exact tag for each model is on its page at **ollama.com/library**.

3 Talk to it once in the terminal, then prove the door Jarvis will use: the OpenAI-compatible endpoint. No key; "ollama" is a placeholder the endpoint ignores.

```
# 2. the brain (pick ONE from the table; this is the 16 GB floor, swap the tag for yours)
ollama pull qwen3.5:9b

# 3a. prove it talks
ollama run qwen3.5:9b "Say hello in one sentence."

# 3b. prove the door Jarvis will use (the same request shape as a cloud brain)
#     Mac / Linux. On Windows run the same in PowerShell with curl.exe and the JSON in a file.
curl http://localhost:11434/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"qwen3.5:9b","reasoning_effort":"none",
       "messages":[{"role":"user","content":"Say hello in one sentence."}]}'

# expect a JSON reply with a "content" string that is NOT empty.
# if "content" is empty and finish_reason is "length", the model spent its whole budget
# thinking silently. "reasoning_effort":"none" is the switch that stops that; keep it.
```

Make it yours: pull two brains and keep both. The 9B for the "good morning" chatter and the biggest one your memory allows for real questions, and Prompt 06 lets you swap between them out loud. Ollama only keeps one loaded at a time, so the first reply after a swap carries a few seconds of loading.

INSTALL 02 · THE VOICE

VoiceStudio and a cloned voice

Best for: the moment Jarvis stops sounding like a computer. Mac and Windows.

1 Install the app with the one-line installer below (Mac in Terminal, Windows in a normal PowerShell), then open it. On the first run click **Install local runtime** and wait; it downloads a few gigabytes once. The engine then answers on port 3900 whenever the app is open. Windows installers are not code-signed yet, so SmartScreen may warn; the script checks the download against the release checksums.

2 In **Model Catalogue**, install a commercially licensed engine and select it. **Mac:** CSM 1B on the MLX audio engine. **Windows (NVIDIA):** VoxCPM2. Both are Apache-2.0, so you may use what they make for anything. If the first-run screen insists on its default voice pack, let it download, then pick your engine anyway; the default pack's weights are licensed for non-commercial use only.

3 Make the voice: **Profiles, New, Clone**, with a 10 to 20 second clip of ONE clean speaker, no music, no room echo. Your own voice is the obvious first clone. Name the profile "Jarvis"; Prompt 02 looks it up by that name.

```
# 1. install. Mac:
curl -fsSL https://voicestudio.sh/install | sh
#    Windows, in PowerShell:
irm https://voicestudio.sh/install.ps1 | iex

# prove the engine is up (the app must be open)
curl http://127.0.0.1:3900/health

# prove the clone speaks: a WAV lands in your folder. Replace PROFILE_ID with the id
# shown by  curl http://127.0.0.1:3900/profiles   and MODEL with your engine:
# Mac  mlx-community/csm-1b-8bit      Windows  voxcpm2
curl http://127.0.0.1:3900/v1/audio/speech -H "Content-Type: application/json" \
  -d '{"model":"MODEL","voice":"PROFILE_ID","language":"en",
       "response_format":"wav","input":"Good afternoon. The local voice is live."}' \
  -o local-voice-check.wav
```

Make it yours: the clip decides everything. Record it on the mic you will actually use, in the room you will actually sit in, reading two or three calm sentences. A shorter clean clip often clones better than a longer one with a breath or a chair creak in it.

INSTALL 03 · THE EARS

Whisper, on your own chip

Best for: closing the one gap most "local" assistants quietly leave open.

1. Install the Whisper package for your machine: **mlx-whisper** on Apple Silicon (fastest), **faster-whisper** on Windows and Linux (it uses an NVIDIA card when there is one and the CPU otherwise). Both are free, open source, and run with no account.
2. Record a five-second test clip of yourself saying a known sentence (Voice Memos, QuickTime or the Windows Sound Recorder, exported as test.wav into your Jarvis folder).
3. Run the proof below. The first run downloads the model once, about 1.5 GB for the large turbo model. The second run is the real speed.

```
# 1. Apple Silicon
python3 -m pip install mlx-whisper

#    Windows, Linux
python3 -m pip install faster-whisper

# 3. prove it hears you. Apple Silicon: expect your sentence printed back.
python3 -c "import mlx_whisper; print(mlx_whisper.transcribe('test.wav',
  path_or_hf_repo='mlx-community/whisper-large-v3-turbo')['text'])"

#    Windows / Linux, same test:
python3 -c "from faster_whisper import WhisperModel; m=WhisperModel('large-v3-turbo');
  print(' '.join(s.text for s in m.transcribe('test.wav')[0]))"

#    the small model Prompt 03 uses for the wake word is 'whisper-small-mlx' on Mac
#    and 'small' with faster-whisper; warm it the same way.
```

Make it yours: if the large model takes more than two seconds on your five-second clip, use the small one for everything. A fast ear that misses a word now and then beats a perfect ear that makes every sentence wait.

Part Three · the prompts

Six prompts for Claude Code, in order. The first three plug the three engines into a Jarvis: the galaxy and the local brain, the cloned voice, the local ears. Then the character, the growing memory, and the spoken brain switch with the drill that proves nothing leaves the machine. This is enough to build a Jarvis you can live with; the rest is yours to add.

The brain	A 3D galaxy of your notes with a brain that answers from them, through Ollama, with the prompt built to be cached.
The voice	Every answer spoken in the cloned voice, chunked so the first words land fast and the rest never pauses mid-sentence.
The ears	Hands-free: "Jarvis" wakes him, Whisper hears the command, interrupt words stop him instantly.
The character	A butler persona under 300 words, because on a local brain every word of the prompt costs time on every turn.
The proof	"Switch to your local brain" out loud, a badge that shows which brain is live, and a Wi-Fi-off drill that lists every outbound connection during a turn.

Two laws hold across all six prompts. Never trade fluency for first-word speed: a voice that starts fast and then pauses between words sounds broken, while one that starts a beat later and flows sounds alive. And every claim is a receipt: Jarvis never says an action happened unless the code handed him proof.

PROMPT 01 · THE FOUNDATION

The Galaxy and the Local Brain

Best for: the first paste. The whole foundation, with a brain that costs nothing, in one pass.

```
Build me a 3D knowledge galaxy from my markdown notes, with a LOCAL brain I can talk to through Ollama. Do the whole thing in one pass, then start it and verify it before you tell me it's done.

MY NOTES: [PASTE THE FULL PATH TO YOUR NOTES FOLDER, or delete this line and write "I have no notes, so generate 30 realistic sample notes about a small business into ./notes first"]
MY BRAIN: [THE OLLAMA TAG YOU PULLED, for example qwen3.5:9b]

THE INDEXER: build.py, Python 3, standard library only. Scan every .md file and write viewer/graph-data.js containing const GRAPH = {nodes: [...], links: [...]}. Each node: a label from the filename, its parent folder as a group, roughly a 700-character excerpt. Link two notes when one mentions the other's title or they share [[wikilinks]]. CRITICAL: every node gets a numeric id equal to its index in the nodes array.

THE VIEWER: viewer/index.html, single page, 3d-force-graph from a CDN pinned to an exact version (Prompt 06 brings it local). No npm, no build step. Cinematic: black space, a starfield, nodes glowing by group, slow idle rotation. Clicking a node flies the camera to it, lights its neighbours, and opens a side panel with the excerpt.

THE SERVER: server.py, port 4700, bound to 127.0.0.1, serving ONLY the viewer/ folder. Create config.json in the PROJECT ROOT, never inside viewer/: {"brain": {"base_url": "http://localhost:11434/v1", "model": "MY BRAIN", "api_key": "ollama", "extra_body": {"reasoning_effort": "none"}}}. Talk to it with the OpenAI-style chat completions request using urllib from the standard library, no SDK, merging extra_body into every request body. The same code must work unchanged against any cloud brain later, by changing only base_url, model and api_key.

THE BRAIN: add POST /chat. Score every note against my question by keyword overlap, weighting title matches higher, take the top 6, and ask the model. System prompt: answer ONLY from these notes, in two or three sentences, and say so plainly when the notes do not cover it. Return {"answer": "...", "nodes": [indexes used]}. Keep a short server-side history so follow-ups work. A clean input bar at the bottom of the viewer shows the answer.

LOCAL BRAIN RULES, because every token costs time here, not cents: the system prompt must be identical on every turn and come FIRST, with the retrieved notes and my question LAST, so the model's cache skips re-reading it. Keep the system prompt under 400 words. Never send a thinking-disabled flag; the extra_body switch is what stops silent thinking. Give the reply at least 600 max tokens and retry once with a bigger budget if the text comes back empty.

FINISH PROPERLY: start the server, confirm the page loads and the graph renders, ask /chat a real question and print the answer AND the seconds it took. Then give me three example questions based on my actual note titles.
```

Make it yours: the seconds printed at the end are your baseline. Write them down, then swap MY BRAIN for a bigger tag and run the same question. That is the only hardware benchmark that matters: your notes, your question, your machine.

PROMPT 02 · SPEECH

The Cloned Voice

Best for: the moment it stops being a website and starts being someone.

```
Give it the cloned voice from VoiceStudio, which is open on this machine and answers on http://127.0.0.1:3900.

MY ENGINE: [Mac: "engine": "mlx-audio", "model_id": "mlx-community/csm-1b-8bit"   Windows: "engine": "voxcpm2",
"model_id": "voxcpm2"]

THE VOICE BAY: voice.py, standard library only, soft-imported by server.py so Jarvis boots fine without it. Add
"voice": {"url": "http://127.0.0.1:3900", MY ENGINE, "profile": "Jarvis", "language": "en"} to config.json. At boot,
and once per engine run, POST /engines/select with {"family": "tts", "backend_id": ENGINE, "model_id": MODEL_ID} so the
engine is pinned and never silently swapped for the app's default. Find the profile id by name from GET /profiles.

THE ENDPOINT: POST /tts with {"text"} returns audio/wav from POST /v1/audio/speech with {"model": MODEL_ID, "input":
text, "voice": PROFILE_ID, "response_format": "wav", "language": LANG}. Cap one request at 600 characters. Cache every
WAV on disk by a hash of text plus profile, so repeated lines like the greeting are instant. Add the header X-Jarvis-
Voice: local:PROFILE_NAME to every reply.

KEEP IT WARM: every 10 idle minutes, render "Standing by." to the engine and throw the audio away, so the model stays
loaded and the first real line of the day is not a cold start.

THE PLAYER, and the law: split each answer into sentence-packed chunks of about 170 characters, never breaking inside a
sentence. Request the first chunk, play it, and fetch the next chunk while the first one plays, so playback never gaps.
Do NOT cut the first chunk shorter to start faster: a voice that pauses in the middle of a sentence sounds broken, and
I will take a beat of extra wait over that every time.

FALLBACK: if GET /health does not answer within one second, speak with the browser's Web Speech API instead and mark
the reply X-Jarvis-Voice: browser. Never fail silently; never crash.

THE MUTE FLAG: if the page URL contains ?mute=1 nothing in that tab ever speaks. Put the check inside the ONE function
that plays audio, so nothing can route around it.

VOICE ROUTES: keep a list of voice routes, each a pattern and a handler, checked BEFORE a sentence goes to the brain.
Add "use your local voice" and "use the browser voice". Later prompts add their own routes to this list.

VERIFY: play a 10-word line and an 80-word answer, print the seconds to first audio for each, confirm the second chunk
was already fetched when the first finished, and confirm a ?mute=1 tab stays silent.
```

Make it yours: clone two profiles, a calm one and an energetic one, and add a route for each. Same brain, different mood, one sentence to switch.

PROMPT 03 · HEARING

The Local Ears

Best for: hands-free, with no audio leaving the room.

```
Give it ears that run on this machine. No browser speech recognition anywhere in this project: it sends audio to a cloud.

THE HEARING BAY: hear.py, using mlx_whisper on Apple Silicon and faster_whisper on Windows and Linux, soft-imported so Jarvis boots without it. Add "hearing": {"model": "large-v3-turbo", "wake_model": "small"} to config.json and map those names to the right repo or size for whichever package is installed. Load both models ONCE at boot, keep them in memory, and warm each with one second of silence so the first real transcription is not a cold start. POST /hear takes a WAV body and returns {"text", "seconds"}.

THE MICROPHONE: in the viewer, capture with getUserMedia and encode 16 kHz mono 16-bit WAV in the browser, no ffmpeg. Measure loudness continuously. An utterance starts when loudness crosses a threshold and ends after SILENCE_MS of quiet, a named constant at the top of the file, starting at 700. Ignore anything under 300 ms. Then apply the FINISH WINDOW: buffer the transcript and wait FINISH_MS (start at 900) for more speech before dispatching, because I pause in the middle of sentences. More speech appends and restarts the timer.

THE WAKE WORD: a hands-free toggle. When it is on, every utterance goes to the SMALL model first. If the text starts with "Jarvis" (also "hey Jarvis"), strip it and send the rest to the LARGE model and on to /chat. If nothing follows the name, say "Yes, sir?" and treat the next utterance as the command. Anything without the name is discarded, never logged, never sent anywhere. A push-to-talk button bypasses the wake word and goes straight to the large model.

INTERRUPTING: while Jarvis is speaking keep listening, but only for "stop" and "wait", which stop playback instantly. Raise the threshold while audio plays so his own voice through the speakers does not trigger it, and tell me plainly if my setup needs headphones.

STATUS: a status line that shows listening, heard (with the text), thinking and speaking, so I always know which of us the machine thinks is talking. ?mute=1 tabs still hear, they just never speak.

VERIFY: a test plan with what to say and what I should see, including one mid-sentence pause to prove the finish window, one sentence without the wake word that must be discarded, and the measured seconds from the end of my speech to the transcript on screen.
```

Make it yours: SILENCE_MS and FINISH_MS are the two most personal numbers in the build. Tune them after a day of real use, never in one sitting, and tune them separately: one is how you pause, the other is how you think.

PROMPT 04 · CHARACTER

The Butler, Lean

Best for: a personality that fits in a small brain and a small prompt.

```
Give it a personality that a local model can carry and a local prompt can afford.

Rewrite the /chat system prompt so the assistant is a dry, impeccably polite British butler with a razor wit. Rules:
- Address me as "sir", occasionally, not in every sentence.
- Answer in ONE witty sentence plus the facts. Never recite the note back; it is on my screen.
- When my notes do not cover something, say so plainly and with a little dignity. Never invent a source.
- Never claim an action happened unless the code actually did it and handed you a receipt. "Done" when nothing happened
is the one unforgivable sin.
- When asked what he costs to run, the honest answer is nothing: he lives on this machine.

THE BUDGET: the entire persona block must be under 300 words, because on a local brain every word of the prompt is paid
for in time on every turn. Smaller models follow short, concrete rules and ignore long ones, so prefer five sharp rules
over fifteen careful ones. Put the persona in ONE clearly commented block at the top of server.py.

STABLE PREFIX: the persona, the rules and the control tags never change between turns and always come first. Anything
that changes (the date, the retrieved notes, my question) comes after them. This is what lets the model reuse its cache
instead of re-reading the whole prompt every time.

BOOT GREETING: when the page loads it says one short line, "Hello, sir.", and nothing more. Show the real indexed note
count ON SCREEN, read from the graph data, never hardcoded.

CONTROL TAGS: when I ask for an action the brain cannot do by talking, it replies with a short tag in square brackets
instead of prose, and the server executes the tag. Keep the tag list in the persona block; later prompts add their own.
Test that this model actually emits the tag reliably: ask five times, count the hits, and if it misses more than once,
simplify the instruction until it does not.

VERIFY: print the token count of the system prompt and the seconds per reply BEFORE and AFTER this change, then give me
five things to say that show off the personality, including one question it cannot answer from my notes.
```

Make it yours: a small brain is a worse actor than a big one, so give it fewer lines. One perfectly dry sentence per answer is a character; three attempted jokes is a model trying too hard. If you moved up to a 27B or a 120B brain, you can afford a sixth rule. Not a sixteenth.

PROMPT 05 · GROWTH

Total Recall

Best for: the point where the brain grows without you opening an editor.

```
Let me grow the brain by voice.

- When I say or type anything starting with "remember that", POST it to a new /remember endpoint that writes a real markdown file into a captures/ folder inside my notes directory, with a sensible title from the first few words and today's date inside the file.
- Add the new node to the running galaxy LIVE, without a page reload: born at the position of its most related existing node, a brief glow pulse, then fly the camera to it.
- Link it into the graph the same way build.py would, so it is a real citizen of the galaxy and can be a source in the very next answer.
- Confirm out loud in one line, in character, in the cloned voice.

ONE DOOR FOR NEW NOTES: put the server side in ONE function (write the file, append the node to graph-data.js, add it to the live search index, return its index) and the viewer side in ONE function (addNote: birth, link, pulse, fly). Every future feature that creates a note goes through exactly these two.

MAKE THE ANSWER PROVE ITSELF: /chat already returns the indexes it used. Fly the camera to the top source node and open its panel while the answer is spoken, so I can see which note it came from. If the answer drew on four or more notes, light the whole cluster instead of flying to one. Small talk never moves the camera: decide whether the question was about my notes BEFORE you decide whether to move anything.

Two things to get right, because they bite later:
1. Writing a file is not the same as indexing it. The new note must be searchable by /chat in the next question without re-running build.py.
2. Never let a capture silently fail. If the write fails for any reason, say so out loud. A second brain that quietly forgets is worse than no second brain.

Then have me test it: I will say "Jarvis, remember that the finish window should be 900 milliseconds", you show me the new star being born, and I will immediately ask a question that should retrieve it.
```

Make it yours: add trigger phrases that match how you actually talk: "note to self", "don't let me forget", "log this". The capture only works if the phrase is one you would say without thinking.

PROMPT 06 · THE PROOF

The Switch and the Wi-Fi-Off Drill

Best for: swapping brains out loud, and being able to say "nothing leaves this machine" and mean it.

```
Let me swap brains by voice, always see which one is live, and prove a conversation never leaves this machine.

THE SHELF: config.json gets a "brains" block of named entries, each with base_url, model, api_key and optional
extra_body. Ship it with "local" (the Ollama entry from Prompt 01), "big" (a second Ollama tag I will fill in, for the
largest brain my memory allows) and "claude" ({"base_url": "https://api.anthropic.com", "model": "claude-opus-5-5",
"api_key": "PUT-YOUR-KEY-HERE"}). The Anthropic entry uses the Messages API shape; keep the two request builders side
by side in one small function. If the key is still the placeholder, "switch to Claude" answers in one dry line that the
cloud brain has no key, and stays local. I will type the real key into the file myself; never ask me for it in chat.

THE ROUTES: "switch to your local brain", "switch to your big brain", "switch to Claude", "which brain are you on".
Swaps are runtime only: every restart boots on the brain named in config.json "boot_brain", default "local". Confirm
each swap in one short line, spoken by the NEW brain, and say which machine it lives on. When a request needs the cloud
and the cloud is unplugged, Jarvis says so in one line instead of pretending; add that rule to the persona block. When
I switch to a brain Ollama has not loaded yet, say "one moment, sir" while it loads.

THE BADGE: a small line under the status that shows BRAIN and VOICE, for example BRAIN local qwen3.5:9b, VOICE local
Jarvis. Hidden by default, visible for 5 seconds whenever either one actually changes, including when the voice falls
back to the browser because the engine died.

BRING THE LIBRARIES HOME: the viewer still loads its 3D libraries from a CDN. Download the exact pinned versions once
into viewer/vendor/, rewrite the imports to those local files, remove every remaining reference to any CDN, and record
the versions and source URLs in viewer/vendor/VERSIONS.txt. After this, the page must render with no network at all.

THE DRILL, as drill.py: wrap the server's outbound HTTP so every connection it opens is logged with the host and the
reason, and read the browser's resource timing entries after a turn. Then: send one real question through /chat, speak
the answer through /tts, feed that WAV back through /hear, and PRINT every host that is not localhost or 127.0.0.1. The
drill passes only when that list is empty. If "switch to Claude" is active with a real key, the drill must FAIL and
name api.anthropic.com, which proves the watcher works. End by printing THE HONEST LIST: the features that still need
the internet, read from the code, not from memory.

VERIFY: in a ?mute=1 tab run every route and show the badge appear and disappear, prove a restart boots back to the
configured brain, grep the viewer and the network log for the placeholder key, run the drill in local mode and paste
the empty host list. Then give me a four-step script for the on-camera version: turn off Wi-Fi, reload the page, ask
one question whose answer lives in one note, and what I should see and hear.
```

Make it yours: the on-camera version is the whole argument in fifteen seconds. Wi-Fi icon crossed out, a question, the galaxy flying to the note, the answer in a cloned voice. And the "big brain" entry is where the memory you paid for shows up: ask both brains the same five questions and the difference is your receipt.

Where to go from here

Read Part One before you buy. Do the three installs in an evening, run the six prompts over a weekend, and live with the offline butler for a week before you judge him. He is simpler than the cloud one. He is also yours: no bill, no account, no audio in anyone's data centre. The first time you ask a question with the Wi-Fi off and the galaxy flies to the note, you will understand why that matters.

1 **Post your machine and your brain.**
The free community has a thread for every pack. Your computer, its memory, the Ollama tag you settled on, and how it feels to live with. That thread becomes the buying guide for everyone after you, and it is more honest than any spec sheet.

2 **Fill the "big" shelf.**
If you bought the memory, use it. Pull the largest model from the table in Part One, put it on the shelf next to the small one, and ask both the same five questions. The difference tells you exactly what the extra memory bought.

3 **Build the thing a cloud assistant cannot be.**
Private by construction. A Jarvis that reads your contracts, your client notes or your medical folder and never sends a byte anywhere is a product in its own right, for a whole class of businesses that are not allowed to use the cloud ones.

Want the finished Jarvis without building it?

Inside the paid AI Workshop community you get the full Jarvis build I run every day, with a five-minute install and no code, plus the classroom, the live calls, and a team that helps when something breaks.

Free community: skool.com/aiworkshop-lite
Full workshop: skool.com/aiworkshop
YouTube: Zubair Trabzada | AI Workshop